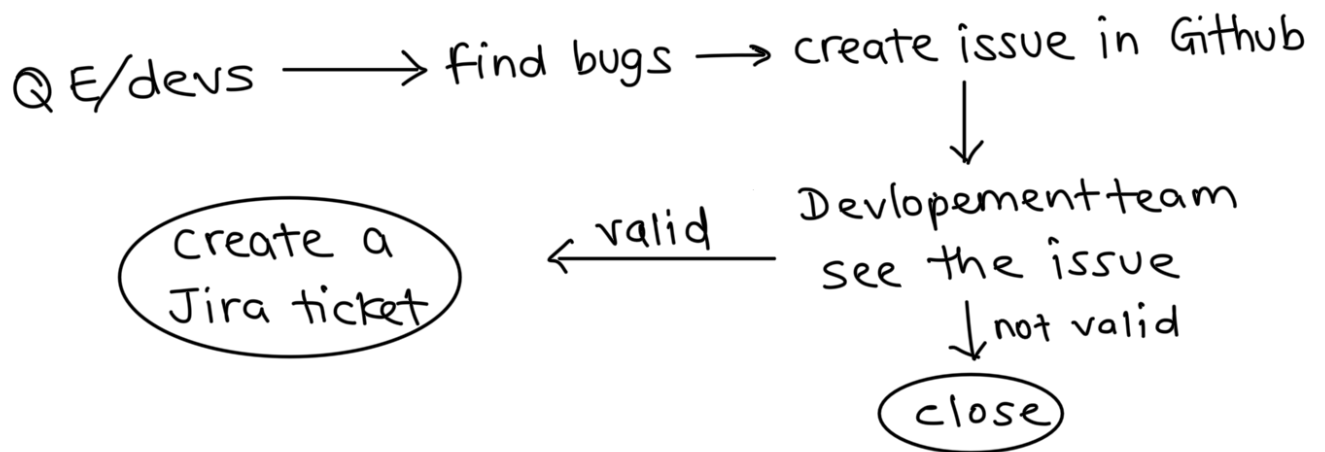


* Github and JIRA Integration project

Problem statement:

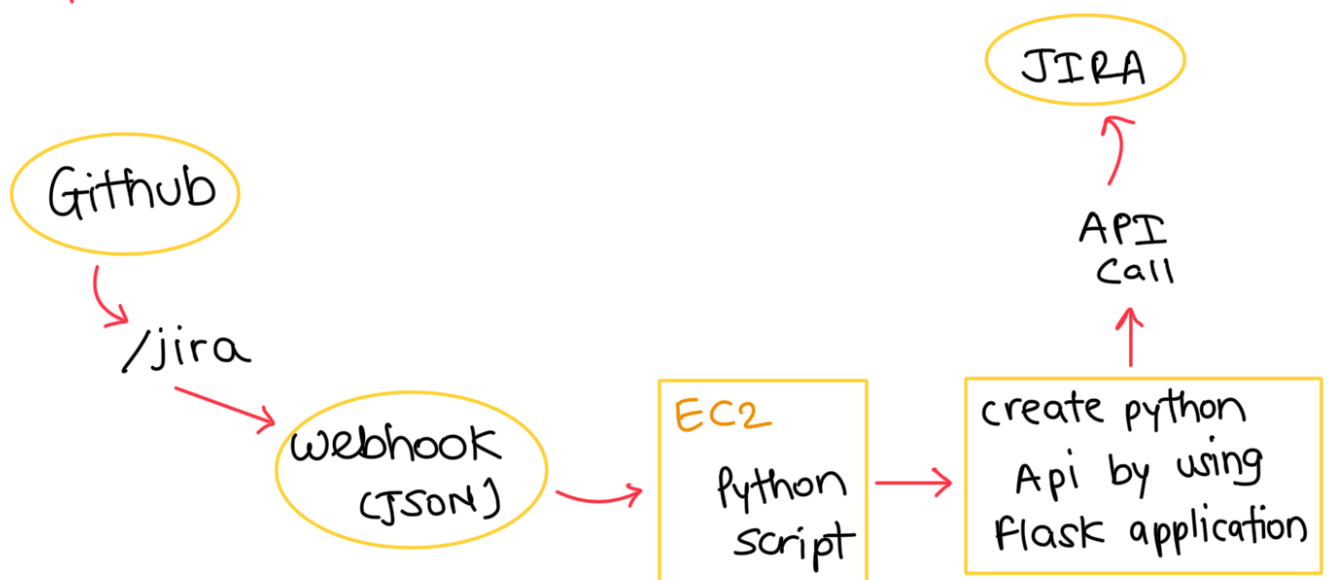


To create a jira ticket developes has login to jira dashboard then create issue write everything.

What we have to do:

Dev comes to devops team and say if issue is valid if simply i write /jira and commit create an jira ticket includes all info.

Steps:



1) Jira Setup

Go to google type jira install → Go Atlassian →
sign in → jira software → Get it for free
→ login → write anything in your site → select
Scrum → write anything in Name and Key

2) Create API token because we connect jira by API

Go to profile → Manage Account → security →
Create and manage API token → Give Name

3) i) Go Jira API documentation search for projects
→ list projects take python-script and paste
in vs code in url replace domain by ours ex.
nkudale939.atlassian.net → also create API-TOKEN
variable and paste API token → go to terminal
run `python <filename>.py` it will give whole JSON
How if we want only name then write
`print(json.loads(response.text)[1]["name"])`

ii) Now we want to create an issue
→ similar like above search for create issue
and paste code in vs code.

→ Delete optional fields

→ Give issuetype id as 10011 (How to find
go to projects setting → configure boards
click on story → in url you see id at end
in project give "key" = "your project key")

4) We create python API because github want to

connect and invoke the script. For this we have to deploy the flask application on ec2 server.

- 5) Now create EC2 instance of ubuntu image
install python, install flask
create demo.py file and write below

```
from flask import Flask
#creating a flask app instance
app=Flask(__name__)
@app.route("/") # decorator
def hello():
    return "hello World"
app.run('0.0.0.0')
esc :wq
python3 demo.py
on browser paste the ip of instance:5000/
we get output
```

} This is example

- 6) Now create another create-jira.py file

```
from flask import Flask, request, jsonify
import requests
from requests.auth import HTTPBasicAuth
import json

app = Flask(__name__)

@app.route("/createJIRA", methods=['POST'])
def createJIRA():
    data = request.json # Extract webhook payload

    # Safely extract the comment body
    comment_body = data.get("comment", {}).get("body", "").strip()
    print(f"Received GitHub comment: {comment_body}")

    if comment_body == "/jira":
        print("Trigger matched: Creating Jira ticket...")

        url = ""

        API_TOKEN = ""
        auth = HTTPBasicAuth("email@gmail.com", API_TOKEN)
```

```
headers = {
    "Accept": "application/json",
    "Content-Type": "application/json"
}
```

```
payload = json.dumps({
    "fields": {
        "description": {
            "content": [
                {
                    "content": [
                        {
                            "text": "Automatically created from GitHub comment /jira",
                            "type": "text"
                        }
                    ],
                    "type": "paragraph"
                }
            ],
            "type": "doc",
            "version": 1
        },
        "issuetype": {
            "id": "10011"
        },
        "project": {
            "key": "AB"
        },
        "summary": "Jira ticket created from GitHub comment"
    },
    "update": {}
})
```

```
response = requests.request(
    "POST",
    url,
    data=payload,
    headers=headers,
    auth=auth
)

return jsonify({"status": "Jira ticket created", "response": response.json()}), 201

else:
    print("Comment did not match '/jira'. Ignoring.")
    return jsonify({"status": "No action taken"}), 200

if __name__ == "__main__":
    app.run('0.0.0.0', port=5000)
```

7) Now Create Webhook

create repo → go to github repo setting → add webhook

→ in payload url paste `http://<public IPV4 DNS of ec2 instance>:5000/createJIRA` (post method)

Select content type as application JSON

Which events would you like to trigger this webhook?

→ select let me select individual events.

select issue comments

create webhook

if error occurs could not connect run the python

Flask application on instance and again do

redeliver

8) Now create some fake issue in github for checking and check.